CS 61C Summer 2024

Andrew, Eddy, Jedi, Nikhil Final

Print Your Name: _____

Print Your Student ID: _____

You have 170 minutes. There are 11 questions of varying credit (100 points total).

Question:	1	2	3	4	5	6	7	8	9	10	11	Total
Points:	15	9	8	8	14	5	6	14	12	9	0	100

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (completely unfilled)
- ● Only one selected option (completely filled)
- ◍ Don't do this (it will be graded as incorrect)

For questions with **square checkboxes**, you may select one or more choices.

- ☐ You can select
- ■ multiple squares
- ■ (completely filled)

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation.

A list of statements that apply to the entire exam is on the last page.

Write the statement below in the same handwriting you will use on the rest of the exam.

> I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat; if these answers are not my own work, I may be deducted up to 0x0123 4567 89AB CDEF points.

_____

_____

_____

_____

_____

Sign your name: _____

Q1 *Potpourri* **(15 points)**

Q1.1 (1.5 points) Convert the unsigned number 2101_3 to 8-bit hexadecimal.

`0x`

Q1.2 (1.5 points) Convert the decimal number -33 to 8-bit hexadecimal.

`0x`

Q1.3 (1 point) Return of attack of the flippy bit! Convert the 8-bit hexadecimal `0xEB` to binary.

`0b`

Q1.4 (2 points) Given an IEEE-754 standard floating point representation with 10 bits: 1 sign bit, 4 exponent bits (with a standard bias of -7), and 5 significand bits, how many NaNs can be represented?

Q1.5 (2 points) Convert the following RISC-V machine code into its corresponding instruction. Express immediates in decimal and use the appropriate register names instead of numbers (i.e. `s5` instead of `x21`).

`0x00F9 0E63`

Q1.6 (1 point) True or False: Given the single-cycle datapath, PCSel should be 1 for all branch instructions.

○ True ○ False

Q1.7 (1 point) True or False: Given the 5-stage pipelined datapath with double-pumping and **no** forwarding, all data hazards will require at most 2 stalls.

○ True ○ False

Q1.8 (1 point) True or False: Pipelining increases the clock frequency by decreasing instruction latency.

○ True ○ False

Q1.9 (1 point) True or False: Stack-allocated variables declared within a `#pragma omp parallel` section are shared amongst all threads.

○ True ○ False

Q1.10 (1.5 points) Suppose that a memory system with the following hit time and hit rates has an AMAT (average memory access time) of 97ns. What is the hit time of DRAM?

	Hit Time	**Hit Rate**
L1 Cache	10ns	70%
L2 Cache	50ns	80%
DRAM	?	100%

ns

Q1.11 (1.5 points) Suppose we have a program that takes 100 minutes to run. If 50% of the code can be sped up by a factor of 5, what is the runtime of the optimized program?

minutes

Q2 *Cones* **(9 points)**

Implement `count_ones`, a RISC-V function, defined as follows:

`count_ones`: Count the number of `1`'s in a number's binary representation		
Arguments	`a0`	A 32-bit number.
Return value	`a0`	The number of `1`'s in `a0`'s binary representation.

For example, calling `count_ones` on `0b1000 0110 0001 1100 1011 1110 1110 1111` would return `19` in `a0`.

```
1  count_ones:
2      beq a0 x0 zero_case
3      _____ Q2.1
4      _____ Q2.2
5      _____ Q2.3
6      slt s7 _____ Q2.4
7      _____ Q2.5
8      jal ra count_ones
9      add a0 _____ Q2.6
10     _____ Q2.7
11     _____ Q2.8
12     _____ Q2.9
13     jr ra
14 zero_case:
15     li a0 0
16     jr ra
```

This page intentionally left (mostly) blank.

The exam continues on the next page.

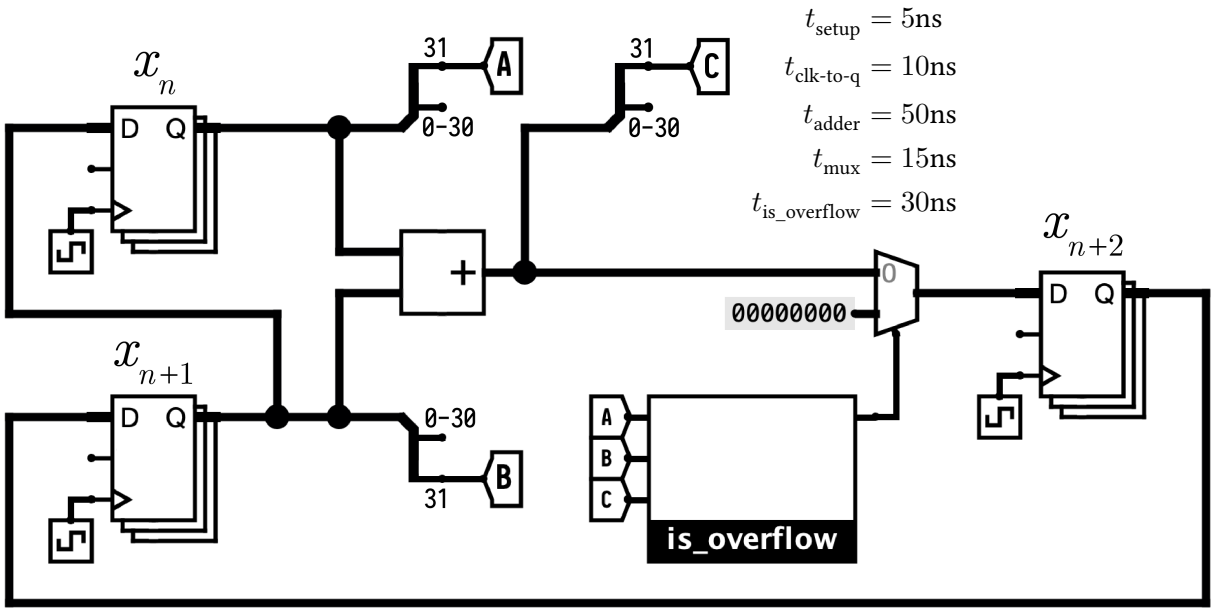
Q3 *Jeddy Sequence* **(8 points)**

Suppose an Eddy sequence of integers is $x_{n+2} = x_{n+1} + x_n$. For example, given $x_0 = 0$ and $x_1 = -1$, then the Eddy sequence would be as follows:

x_0	x_1	x_2	x_3	x_4	x_5	x_6	$\cdots$
0	-1	-1	-2	-3	-5	-8	$\cdots$

Jedi built the following circuit to calculate the next number in an Eddy sequence. At $t = 0$, assume register x_n has value x_0, register x_{n+1} has value x_1, and register x_{n+2} has value x_2. Assume components have the combinational logic delays listed, and other components have negligible delay.

The `is_overflow` subcircuit outputs a `1` if the adder overflows and `0` otherwise. If the adder overflows, the circuit should store `0x00000000` in the x_{n+2} register.



Q3.1 (2 points) What is the **minimum clock period** for this circuit to function properly, in nanoseconds?

ns

Q3.2 (2 points) What is the **maximum hold time** the registers can have such that there are no hold time violations, in nanoseconds?

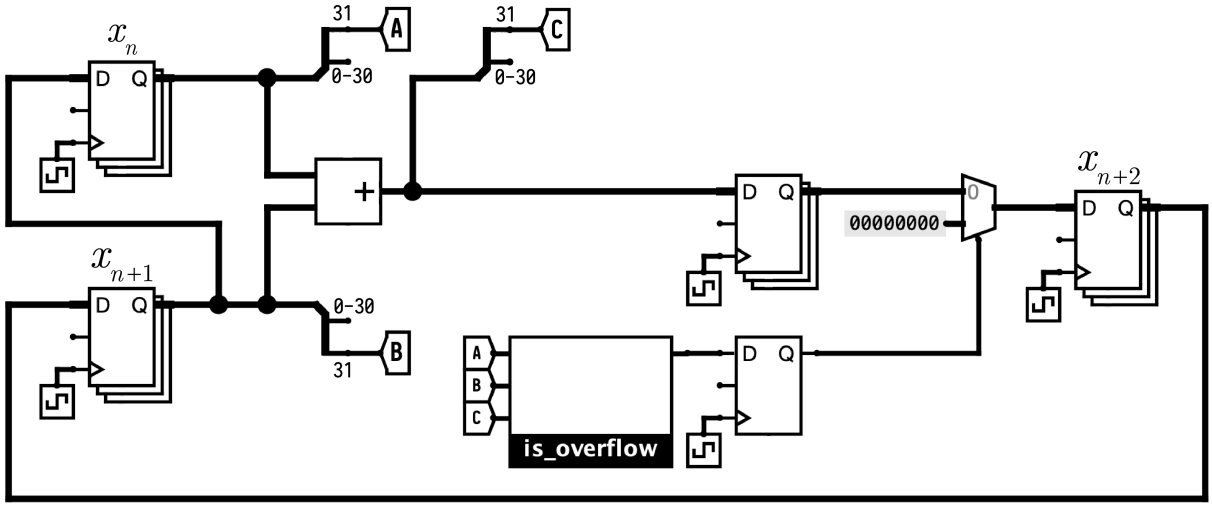
ns

Q3.3 (2 points) Write a simplified Boolean expression that describes the behavior of the `is_overflow` subcircuit. You may use at most 7 operators. NOT (`!`), AND (`*`), OR (`+`) each count as one operator. We will assume standard C operator precedence, so use parentheses when uncertain.

Your answer may consist of the following:

Inputs	NOT	AND	OR	Constants	Parentheses
`A`, `B`, `C`	`!`	`*`	`+`	`0`, `1`	`()`

Jedi wants to generate Eddy sequence numbers faster by pipelining the circuit! The proposed pipeline registers are shown in the diagram below, dividing the circuit into two stages.



Q3.4 (2 points) True or False: Will the added pipeline registers improve the maximum throughput in additions/cycle of the circuit? Briefly explain your answer in 15 words or less.

○ True ○ False

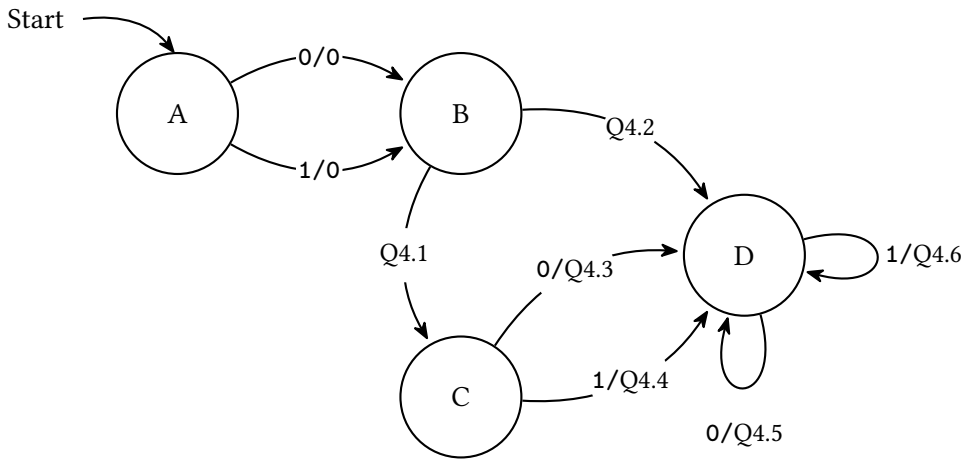
Q4 *iFPSM* **(8 points)**

The iFPSM is a finite state machine that identifies if a floating point number is $+\infty$ or $-\infty$. The IEEE-754 standard representation of the floating point number will be passed bitwise as the input into the iFPSM starting from the MSB (the sign bit).

Input	Output
$+\infty$ or $-\infty$	An all zero bitstring
Non-infinite value	Any non-zero bitstring

Below is an iFPSM for the IEEE-754 standard floating point representation with 6 bits: 1 sign bit, 2 exponent bits (with a standard bias of -1), and 3 significand bits.

For Q4.1 – Q4.6, select the state transitions such that the iFPSM matches the described behavior.



Q4.1 (1 point)

- ○ 0/0
- ○ 0/1
- ○ 1/0
- ○ 1/1

Q4.2 (1 point)

- ○ 0/0
- ○ 0/1
- ○ 1/0
- ○ 1/1

Q4.3 (0.5 point)

- ○ 0
- ○ 1

Q4.4 (0.5 point)

- ○ 0
- ○ 1

Q4.5 (0.5 point)

- ○ 0
- ○ 1

Q4.6 (0.5 point)

- ○ 0
- ○ 1

Let's extend our iFPSM to be compatible with any number of floating point bits.

Assume we are working with an IEEE-754 standard floating point representation with 1 sign bit, e exponent bits (with a standard bias of b), and s significand bits.

Q4.7 (2 points) What is the fewest number of states that a correct iFPSM must have? Your answer may include e, b, and/or s.

For example, the iFPSM from Q4.1 – Q4.6 has 4 states: A, B, C, and D.

	states

Q4.8 (2 points) The nFPSM is a finite state machine like the iFPSM except it identifies if a floating point number is NaN.

Input	Output
NaN	An all zero bitstring
Non-NaN value	Any non-zero bitstring

What is the fewest number of states that a correct nFPSM must have? Your answer may include e, b, and/or s.

	states

Q5 *Jump and Lychee* **(14 points)**

For the entirety of this question, assume we are working with the single-cycle datapath.

A `ja` instruction in RISC-V is a new instruction described as follows:

`ja rd rs1 imm`

```
1 PC = rs1 & imm
2 rd = PC + 4
```

For each of the following control signals, indicate the value it should always have for `ja`.

Q5.1 (1 point) PCSel

- ○ `0`
- ○ `1`
- ○ None of the above

Q5.2 (1 point) ASel

- ○ `0`
- ○ `1`
- ○ None of the above

Q5.3 (1 point) BSel

- ○ `0`
- ○ `1`
- ○ None of the above

Q5.4 (1 point) ALUSel

- ○ `add`
- ○ `sub`
- ○ `mul`
- ○ `or`
- ○ `and`
- ○ None of the above

Q5.5 (1 point) MemRW

- ○ Memory read
- ○ Memory write
- ○ None of the above

Q5.6 (1 point) WBSel

- ○ `0`
- ○ `1`
- ○ `2`
- ○ None of the above

Q5.7 (1 point) RegWEn

- ○ Register write
- ○ Register read
- ○ None of the Above

Given a new instruction `lieqi` with the following description:

`lieqi rd rs1 rs2 imm`

```
1 if (rs1 == rs2) {
2     rd = imm
3 }
```

For Q5.8 – Q5.9, write a simplified Boolean expression that describes the given control logic signal for `lieqi`. Your answer may consist of the following:

NOT	AND	OR	Constants	Parentheses	Variables
`!`	`*`	`+`	`0, 1`	`()`	Any control logic input or output signal

Q5.8 (1.5 points) PCSel =

Q5.9 (1.5 points) RegWEn =

Q5.10 (4 points) What modifications must be made to the single cycle datapath with as few changes as possible? Select all that apply.

- ☐ Create a new instruction type and update the ImmGen.
- ☐ Add a new read input to the RegFile for a third register value.
- ☐ Add a new WriteData and WriteIndex input to the RegFile.
- ☐ Add a new input to the AMux and update any relevant selector/control logic.
- ☐ Add a new input to the BMux and update any relevant selector/control logic.
- ☐ Add a third input into the ALU and update any relevant selector/control logic.
- ☐ Allow the ALU to send out more than 1 output and update any relevant selector/control logic.
- ☐ Add a new ALU operation and update any relevant selector/control logic.
- ☐ Allow the DMEM to be able to read and write at the same clock cycle and update any relevant selector/control logic.
- ☐ Add a new WBMux and update any selector/control logic.
- ☐ None of the above

Q6 *Hazardous Waters* **(5 points)**

For the entirety of this question, assume that we are using the 5-stage pipelined datapath with double pumping and **no** forwarding.

Q6.1 (3 points) Rearrange the following instructions to minimize the number of stalls while maintaining the same behavior.

```
1 lw t0 0(s0) # Instruction A
2 sw t0 0(s1) # Instruction B
3 lw t1 4(s0) # Instruction C
4 sw t1 4(s1) # Instruction D
5 lw t2 8(s0) # Instruction E
6 sw t2 8(s1) # Instruction F
```

Format your answer as a comma-separated list. For example, the instruction order in the code block above would be described by "A, B, C, D, E, F".

Q6.2 (2 points) Consider a new instruction, `jellyfish`, that writes the output of the ALU to the RegFile during the EX stage. You may assume that the 5-stage pipelined datapath has been modified to implement this instruction without resolving any potential hazards.

Which hazard would the `jellyfish` instruction cause in the following program?

```
1 addi s0 x0 1
2 addi t4 x0 5
3 jellyfish t0 t1 t2
```

- ○ Structural
- ○ Control
- ○ Data
- ○ None of the above

Q7 *Person-Level Parallelism* (6 points)

Riley's emotions need help to keep everything running and complete the following tasks:

Task Number	Task Name	Time (hours)	Prerequisites
0	Analyze new feelings	3	-
1	Build headquarters	4	-
2	Create memories	2	0
3	Direct emotions	5	1
4	Express joy	4	2, 3
5	Form an identity	2	1, 2
6	Grow friendships	3	4, 5
7	Highlight happy moments	3	6

Q7.1 (2 points) What is the minimum time it takes for Joy to do all the tasks, assuming she can only work on one task at a time?

hours

Joy needs some help, so she asks Nostalgia to help her.

For Q7.2 – Q7.3, assume both Joy and Nostalgia are working on the tasks, each can work on their own task simultaneously, and each can work on one task at a time.

Q7.2 (2 points) What is the minimum time it takes for Joy and Nostalgia to complete all the tasks?

hours

Q7.3 (2 points) Suppose Riley's emotions have a new task as follows:

Task Number	Task Name	Time (hours)	Prerequisites
8	Inspire belief	1	4

What is the minimum time it takes for Joy and Nostalgia to do all the tasks?

hours

Q8 *(Delayed|Train)-Level Parallelism* **(14 points)**

BART trains often arrive much later than their expected arrival times. The `total_delay` function is defined as follows:

`total_delay`: Calculates the sum of all delays between a set of expected and actual arrival times.		
Arguments	`int32_t n`	The number of trains. You may assume `n` is a multiple of 4.
	`int32_t *expected`	An array of `n` expected arrival times.
	`int32_t *actual`	An array of `n` actual arrival times.
Return value	`int32_t`	The sum of all delays.

If a train arrives earlier or at its expected time, the delay should be 0. You may also assume that a train expected to arrive at `expected[i]` actually arrives at `actual[i]`.

For example, given `n = 4` and the following `expected` and `actual` arrays, the return value should be `5 + 3 + 0 + 0 = 8`:

```
expected:  [10, 20, 30, 40]
  actual:  [15, 23, 29, 40]
   delay:  [ 5,  3,  0,  0]
```

You may use the following SIMD operations, with a maximum of one SIMD operation per blank. A `vector` is a 128-bit vector register capable of holding 4 32-bit signed integers.

- `vector vec_load(int32_t *A)`: Loads 4 integers at memory address `A` into a vector
- `vector vec_setnum(int32_t num)`: Creates a vector where every element is equal to `num`
- `vector vec_cmpgt(vector A, vector B)`: Computes whether each element in `A` is greater than the corresponding element in `B`, and returns a new vector with the results (`0xFFFFFFFF` if true, `0` otherwise, for each element in the vector)
- `vector vec_and(vector A, vector B)`: Computes the bitwise `and` between each pair of corresponding vector elements in `A` and `B`, and returns a new vector with the result
- `vector vec_or(vector A, vector B)`: Computes the bitwise `or` between each pair of corresponding vector elements in `A` and `B`, and returns a new vector with the result
- `vector vec_add(vector A, vector B)`: Adds `A` and `B` together elementwise, and returns a new vector with the result
- `vector vec_mul(vector A, vector B)`: Multiplies `A` and `B` together elementwise, and returns a new vector with the result
- `int32_t vec_sum(vector A)`: Adds all elements of the vector together, and returns the sum

Implement `total_delay` to match the described behavior using SIMD and OpenMP.

```
1  int32_t total_delay(int32_t n, int32_t *expected, int32_t *actual) {
2    ____Q8.1____ result = ____Q8.2____;
3    vector zero_vec = vec_setnum(0);
4    vector negate_vec = ____Q8.3____;
5    #pragma ____Q8.4____
6    {
7      vector result_vec = vec_setnum(0);
8      #pragma ____Q8.5____
9      for (int32_t i = ____Q8.6____; ____Q8.7____; i += 4) {
10       vector expected_vec = vec_load(____Q8.8____);
11       vector actual_vec = vec_load(____Q8.9____);
12       vector temp_vec = ____Q8.10____;
13       temp_vec = ____Q8.11____;
14       vector compared_vec = vec_cmpgt(temp_vec, ____Q8.12____);
15       temp_vec = ____Q8.13____;
16       ____Q8.14____;
17     }
18     ____Q8.15____
19     ____Q8.16____
20   }
21   return result;
22 }
```

Q9 *Virtual Memory* **(12 points)**

Q9.1 (1.5 points) Suppose we have 16 MiB of virtual memory, 32 KiB of physical memory, and 64B pages. How many bits are our page offset, virtual page number (VPN), and physical page number (PPN)?

VPN:	PPN:	Offset:

For Q9.2 – Q9.6, assume we have 8-bit page offsets, 24-bit VPNs, and 8-bit PPNs. We have a 4 entry, fully associative TLB with LRU replacement policy and one free physical page with PPN `0x62`.

The current state of the TLB and the first 6 entries of the page table are given below.

TLB

Valid	VPN	PPN
0	0x00 0004	0x28
1	0x00 0001	0x15
1	0x00 0005	0x90
0	0x00 0002	0x46

Page table

0x3040CA80
0xA7708515
0x63A01986
0xBA025068
0xBF586042
0xD5762290
...

Each page table entry (PTE) is 32 bits:

For each of the following virtual addresses, translate it to its corresponding physical address, and answer whether it will result in a TLB hit, TLB miss and page table hit, or Page fault. Assume each access occurs independently, not sequentially.

Q9.2 (1.5 points) `0x0000 0280`

0x

- ○ TLB hit
- ○ TLB miss and page table hit
- ○ Page fault

Q9.3 (1.5 points) `0x0000 03AF`

0x

- ○ TLB hit
- ○ TLB miss and page table hit
- ○ Page fault

Q9.4 (1.5 points) `0x0000 0512`

0x

- ○ TLB hit
- ○ TLB miss and page table hit
- ○ Page fault

Q9.5 (1.5 points) `0x0000 0484`

0x

- ○ TLB hit
- ○ TLB miss and page table hit
- ○ Page fault

Q9.6 (1.5 points) `0x0000 006A`

0x

- ○ TLB hit
- ○ TLB miss and page table hit
- ○ Page fault

For Q9.7 – Q9.8, assume we have a 2-level page table with 16-bit page offsets, 16-bit VPNs, 8-bit PPNs, and no TLB.

Assume that we divide the VPN bits equally between the L1 and L2 page tables, and the L1 VPN bits are the most significant bits.

Q9.7 (1 point) For the address `0xF944 0620`, what are the L1 VPN and L2 VPN?

L1: 0x | L2: 0x

Q9.8 (2 points) How many different L2 page tables are accessed in total over the following sequence of memory accesses?

- `0xFFFF F100`
- `0xFFFF F000`
- `0x1000 8000`
- `0xFFFC 6500`
- `0x4000 8000`
- `0x100A B608`
- `0x10FD 0904`

Q10 *Caches* **(9 points)**

Q10.1 (1.5 points) What is the T:I:O breakdown for a 128B direct mapped cache with a block size of 16B on a 16-bit system?

T: bits	I: bits	O: bits

For Q10.2 – Q10.4, assume we are using a 2-way set-associative cache with an MRU replacement policy, and each address has 11 tag bits, 2 index bits, and 3 offset bits.

```
1 #define ARR_SIZE 100
2 int32_t arr[ARR_SIZE]; // arr starts at address 0x2020
3 for (register int32_t i = 0; i < ARR_SIZE - 5; i++) {
4   arr[i] = arr[i] + 0x6;
5   arr[i + 2] = arr[i] + 0x1;
6   arr[i + 4] = arr[i] + 0xC;
7 }
```

Q10.2 (1.5 points) How many memory accesses are there in each iteration of the `for` loop?

Q10.3 (2 points) How many cache hits are there in the first iteration of the `for` loop (`i = 0`)?

Q10.4 (2 points) How many cache hits are there in the third iteration of the `for` loop (`i = 2`)?

Q10.5 (2 points) For a fully associative cache, what is the maximum block size for which the cache would always miss for the following program? Assume that the block size must be a power of 2.

```
1 #define ARR_SIZE 100
2 int32_t arr[ARR_SIZE]; // arr starts at address 0x8000
3 register int32_t sum = 0;
4 for (register int32_t i = 0; i < ARR_SIZE; i += 32) {
5     sum += arr[i];
6 }
```

bytes

Q11 *The Finish Line* **(0 points)**

These questions will not be assigned credit; feel free to leave them blank.

Q11.1 (0 points) For each rating on a scale of 1 (worst) to 6 (best), list a nearby restaurant that matches the rating.

1:	2:
3:	4:
5:	6:

Q11.2 (0 points) If there's anything else you want us to know, or you feel like there was an ambiguity in the exam, please put it in the box below.

For ambiguities, you must qualify your answer and provide an answer for both interpretations. For example, "if the question is asking about A, then my answer is X, but if the question is asking about B, then my answer is Y". You will only receive credit if it is a genuine ambiguity and both of your answers are correct. We will only look at ambiguities if you request a regrade.

Unless otherwise specified, the following instructions apply to the entire exam:

- You may assume the following:
  - All provided code compiles successfully and runs without errors
  - All necessary standard libraries are imported
  - Dynamic memory allocation never fails
  - Caches and TLB start off cold
  - The single-cycle and the 5-stage pipelined datapaths are referring to the datapaths on the CS 61C Reference Card
  - No compiler optimizations are performed
  - All expressions follow C operator precedence
- Answer Formats
  - If in hexadecimal, use only capitalized letters (`0xDEADBEEF` instead of `0xdeadbeef`)
  - If the answer is in binary or hexadecimal, include prefixes and do not truncate leading 0's
  - If answer is not in binary or hexadecimal, do not add any prefixes or suffixes
  - Answers must be in the simplest form
  - If no base is specified and the answer is a number or mathematical expression, the answer must be in base 10
- Coding Questions
  - Blanks may be left empty, but do not use more blanks than provided
  - Each blank may contain at most one statement or instruction
  - Programs must be as memory efficient as possible
  - In C, curly braces must be included for `if`/`for`/`while`/`do-while`/`switch` statements, and a block's body may not be on the same line as the opening or closing curly brace
    - For example, an `if` statement requires at least 3 lines, and an `if-else` statement requires at least 5 lines
  - In RISC-V, programs must follow proper calling convention